

# Rapport - Détection de tweets suspects

Master Fouilles de Données et Intelligence Artificielle

KAFANDO WEND-NEMI JOSIAS

IDENTIFIANT : N00428220141

## I. Introduction

Ce projet consiste à construire un modèle capable de dire si un tweet est suspect ou non. Les réseaux sociaux servent à beaucoup de choses utiles, mais ils peuvent aussi être utilisés pour diffuser du contenu problématique, et il serait utile de pouvoir repérer ce genre de contenu automatiquement plutôt que de tout vérifier à la main.

Le but de ce travail n'est pas seulement d'entraîner un modèle qui donne un bon score, mais de suivre toutes les étapes d'un vrai projet de machine learning : explorer les données, les nettoyer, choisir une représentation numérique du texte, comparer plusieurs modèles, les évaluer sérieusement, et enfin les rendre utilisables via une petite application. Le tout est aussi suivi avec Git et DVC, pour que quelqu'un d'autre puisse reproduire les résultats.

## II. Méthodologie

### A. Les données

Le dataset contient environ 60 000 tweets, avec pour chacun un texte (message) et une étiquette (label) qui vaut 1 si le tweet est considéré comme suspect et 0 sinon. Après avoir enlevé les lignes vides et les doublons, il reste 59 416 tweets utilisables.

Un point important remarqué dès le début : les classes sont très déséquilibrées. Environ 90% des tweets sont étiquetés suspects et seulement 10% non suspects. Ce déséquilibre a dû être pris en compte lors de l'entraînement des modèles, sinon un modèle pourrait obtenir une bonne accuracy juste en prédisant toujours la classe majoritaire.

### B. Prétraitement du texte

Un tweet contient beaucoup d'éléments qui ne servent pas à comprendre le contenu, et qui peuvent même perturber un modèle. Les étapes de nettoyage appliquées sont, dans l'ordre :

- mise en minuscules
- suppression des URLs
- suppression des mentions (@) et du symbole # (le mot du hashtag est gardé, seul le symbole part)
- suppression des caractères spéciaux et de la ponctuation
- suppression des stop words (mots très fréquents comme the, is, and, qui n'apportent pas de sens particulier)
- lemmatisation, pour ramener chaque mot à sa forme de base (par exemple running devient run)

Ces choix ont été faits pour réduire le bruit dans le texte et garder seulement les mots qui portent vraiment un sens.

### C. Représentation des données

Pour transformer le texte en nombres, le choix s'est porté sur le TF-IDF. Les tweets étant courts, une représentation simple suffit pour capturer l'essentiel du sens, et le TF-IDF a l'avantage de diminuer le

poids des mots qui reviennent dans presque tous les tweets, tout en augmentant celui des mots plus spécifiques. C'est aussi une méthode rapide à calculer, contrairement à des approches comme BERT qui demandent beaucoup plus de ressources. Le vectoriseur a été configuré avec un vocabulaire de 20 000 mots maximum, en gardant aussi les bigrammes (des paires de mots) en plus des mots seuls, pour capturer un peu de contexte.

## D. Modèles utilisés

Trois algorithmes de classification ont été comparés : Logistic Regression, Random Forest et Naive Bayes. Les deux premiers ont été entraînés avec l'option `class_weight="balanced"`, qui donne plus d'importance aux erreurs faites sur la classe minoritaire pour compenser le déséquilibre. Naive Bayes ne propose pas cette option, il a donc été entraîné sans rééquilibrage, uniquement à titre de comparaison.

Pour améliorer le meilleur modèle (Random Forest), une recherche d'hyperparamètres a été faite avec GridSearchCV, en testant plusieurs valeurs du nombre d'arbres (`n_estimators`) et de la profondeur maximale (`max_depth`), avec une validation croisée à 5 blocs.

## III. Résultats

### A. Comparaison des performances

Les trois modèles ont d'abord été comparés sur un simple split train/test (80% / 20%) :

| Modèle              | Accuracy | Précision | Recall | F1-score |
|---------------------|----------|-----------|--------|----------|
| Logistic Regression | 0.976    | 0.980     | 0.993  | 0.987    |
| Random Forest       | 0.983    | 0.985     | 0.996  | 0.991    |
| Naive Bayes         | 0.916    | 0.914     | 1.000  | 0.955    |

Une validation croisée à 5 blocs a ensuite été faite pour vérifier que ces résultats ne dépendaient pas trop du hasard du split. Les scores obtenus sont très proches des précédents, ce qui est plutôt rassurant :

| Modèle              | Accuracy | Précision | Recall | F1-score |
|---------------------|----------|-----------|--------|----------|
| Logistic Regression | 0.973    | 0.978     | 0.992  | 0.985    |
| Random Forest       | 0.981    | 0.984     | 0.994  | 0.989    |
| Naive Bayes         | 0.911    | 0.910     | 1.000  | 0.953    |

Random Forest ressort comme le meilleur modèle dans les deux cas. Naive Bayes a un recall presque parfait mais une précision plus faible, ce qui veut dire qu'il a tendance à classer trop souvent des tweets normaux comme suspects.

La recherche d'hyperparamètres sur Random Forest a trouvé comme meilleure configuration `n_estimators=200` et `max_depth=None` (pas de limite de profondeur), avec un F1-score moyen de 0.989 en validation croisée. Avec cette configuration, sur le jeu de test final, le modèle obtient une accuracy de 0.982, une précision de 0.985, un recall de 0.995, un F1-score de 0.990 et une AUC de 0.949.

## B. Visualisations

La distribution des classes montre clairement le déséquilibre entre les deux classes :

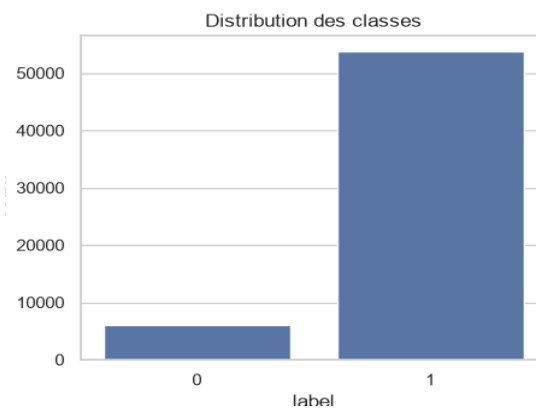


Figure 1 : distribution des classes (suspect / non suspect)

La matrice de confusion et la courbe ROC du modèle final montrent un modèle qui sépare bien les deux classes sur le jeu de test :

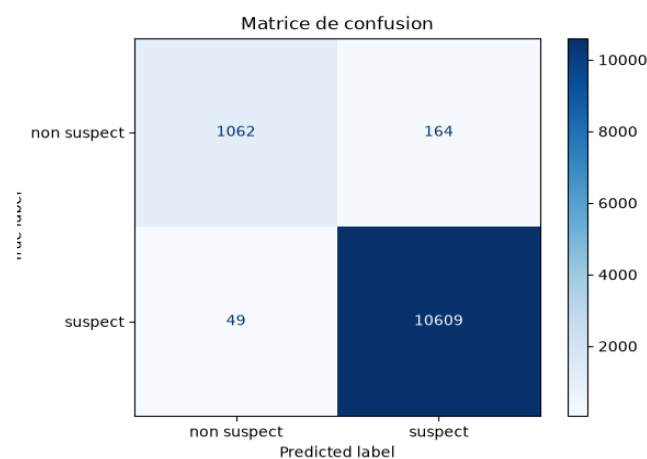


Figure 2 : matrice de confusion (Random Forest, modèle final)

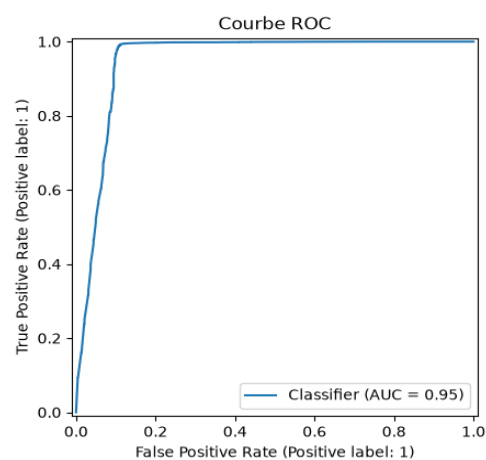


Figure 3 : courbe ROC (Random Forest, modèle final)

## IV. Discussion

### A. Limites du projet

Le modèle obtient de très bons scores sur le jeu de test, mais un test plus informel avec des phrases écrites à la main a révélé quelque chose d'important : le modèle classe presque toujours "suspect" des phrases anglaises tout à fait neutres (par exemple une phrase sur une journée passée avec des amis), même quand tous les mots de la phrase sont bien connus du vocabulaire du TF-IDF.

Ça ne veut pas dire que le pipeline est cassé : sur des exemples réels tirés du jeu de test, le modèle classe bien les deux classes. Mais ça montre que l'étiquette "suspect" de ce dataset ne correspond peut-être pas exactement à ce qu'on imagine intuitivement (un tweet malveillant ou dangereux). En regardant des exemples réels du dataset, les tweets étiquetés "suspects" ne contiennent pas forcément de contenu qui paraît suspect à la lecture. Il est possible que l'étiquette du dataset reflète un autre critère que celui suggéré par son nom, même s'il n'a pas été possible de confirmer avec certitude son origine exacte.

### B. Difficultés rencontrées

La recherche d'hyperparamètres avec Random Forest et une profondeur illimitée s'est avérée beaucoup plus lente que prévu (plusieurs minutes par entraînement), et l'ordinateur utilisé s'est mis en veille plusieurs fois pendant les calculs les plus longs, ce qui a obligé à relancer le calcul à plusieurs reprises.

### C. Perspectives d'amélioration

Plusieurs pistes pourraient améliorer ce travail. Du côté de la représentation du texte, essayer des embeddings plus avancés (Sentence Transformers ou BERT) pourrait mieux capturer le sens des tweets que le TF-IDF. Du côté du déséquilibre des classes, comparer le SMOTE ou l'undersampling avec le class weight utilisé ici permettrait de voir si d'autres stratégies fonctionnent mieux. Enfin, vu la limite observée sur des phrases inhabituelles, il serait intéressant de tester le modèle sur un autre jeu de tweets, différent de celui utilisé pour l'entraînement, pour mieux évaluer sa capacité à généraliser.

## V. Éléments bonus

En plus des parties demandées dans le sujet, trois éléments supplémentaires ont été ajoutés au projet.

### A. Suivi des expériences avec MLflow

En complément du suivi déjà fait avec DVC, les scripts `train.py` et `evaluate.py` enregistrent aussi les paramètres et métriques de chaque exécution avec MLflow (`mlflow.log_params`, `mlflow.log_metrics`). Cela permet de comparer facilement les différentes exécutions du pipeline depuis l'interface MLflow.

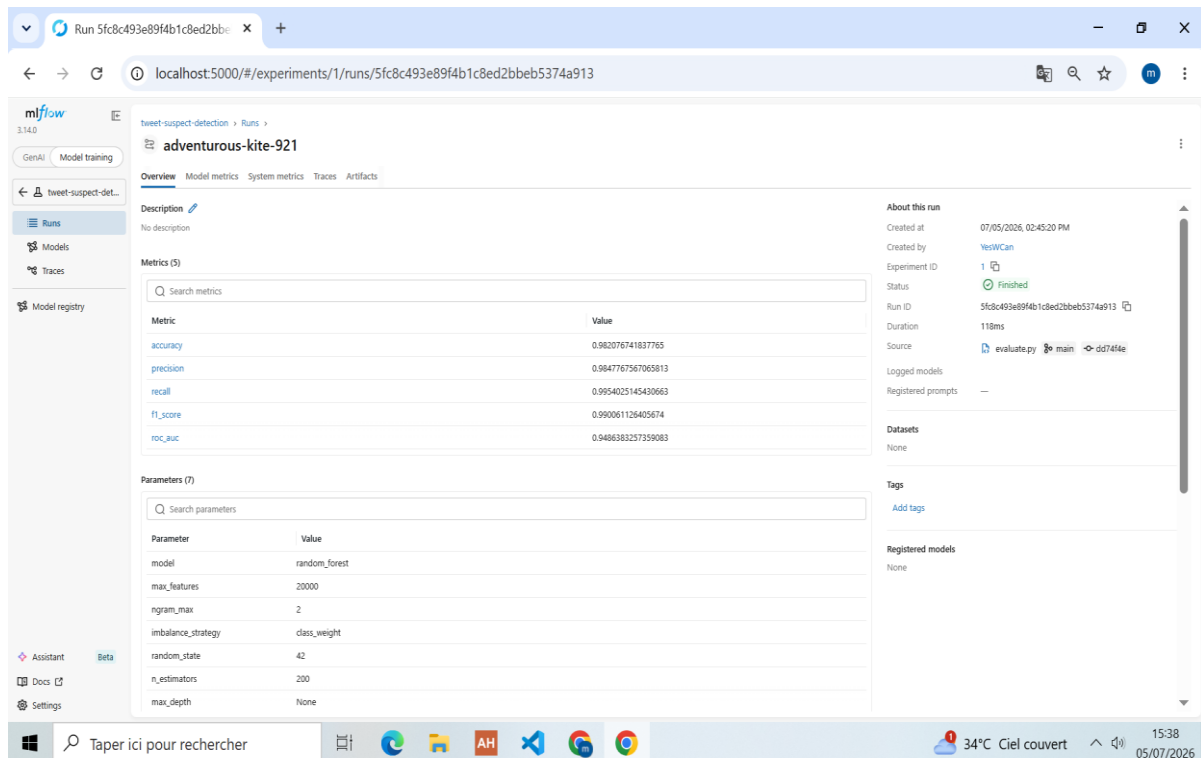


Figure 4 : interface MLflow avec les paramètres et métriques d'une exécution

## B. Déploiement sur le cloud

L'application Streamlit n'est pas seulement lancée en local : elle est aussi déployée publiquement sur Streamlit Community Cloud, à l'adresse suivante :

<https://tweet-suspect-detection-r8udgvufgsjw8mwwwxyuoz.streamlit.app/>

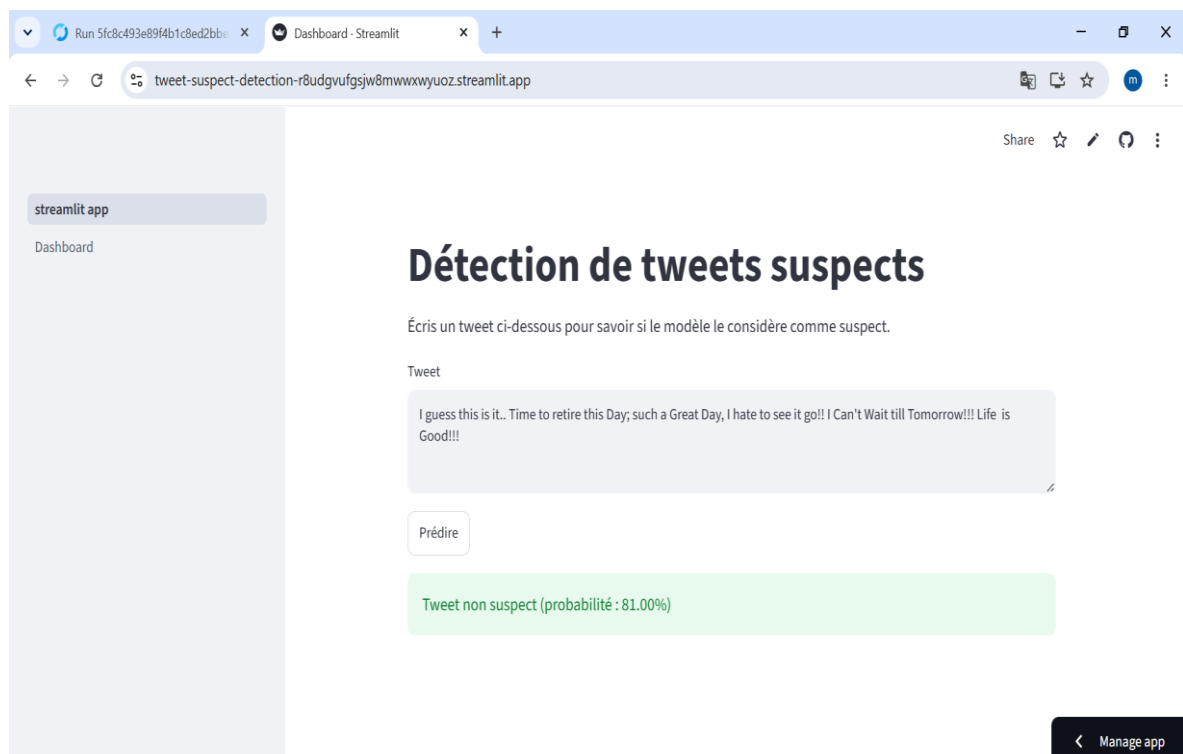


Figure 5 : application déployée sur Streamlit Community Cloud

## C. Tableau de bord de suivi du modèle

Une deuxième page a été ajoutée à l'application Streamlit (accessible via le menu "Dashboard"), qui reprend les métriques du modèle final, la distribution des classes, la matrice de confusion et la courbe ROC, pour pouvoir suivre les performances du modèle sans avoir à rouvrir les notebooks.

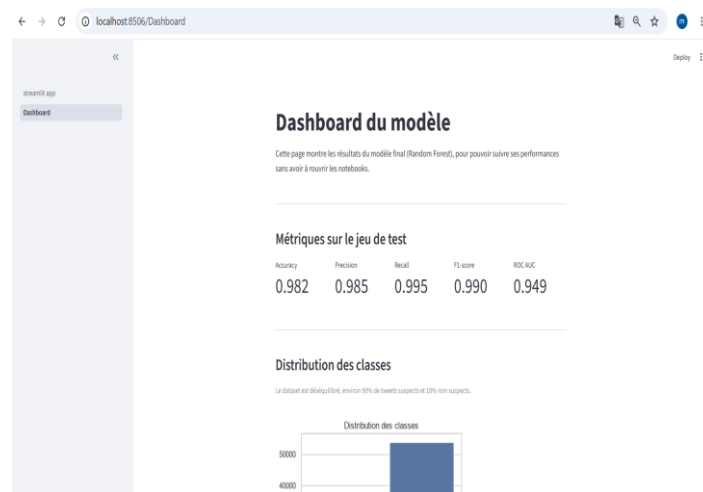


Figure 6 : tableau de bord de suivi du modèle dans Streamlit